

CSSCM 2026 COHORT

TASK 2(a): EULER'S METHOD PROJECT

Numerical Cooling of a Radiating Ball Using Python and Gnuplot

Directors	E. S. Menkah (PhD); E. Adei (PhD)
Tutors	E. Oppong (MPhil); O. Adu-Gyamfi (BSc)

Project Objective

Implement Euler's method in Python for a radiative cooling problem, investigate the effect of step size, write the numerical results to a file, and use Gnuplot to compare two temperature curves on the same chart.

1. Background: Euler's Method

Euler's method is a numerical technique for solving an ordinary differential equation of the form

$$dy/dx = f(x,y), y(0) = y_0$$

using the update formula

$$y(i+1) = y(i) + f(x(i), y(i)) h$$

where h is the step size. The method estimates the next value from the current value and the current slope.

2. Cooling Problem

A ball at 1200 K is allowed to cool in air at an ambient temperature of 300 K. Assuming heat is lost only due to radiation, the temperature is governed by

$$d(\theta)/dt = -2.2067 \times 10^{-12} [\theta^4 - 81 \times 10^8]$$

with

$$\theta(0) = 1200 \text{ K}$$

The objective is to estimate the temperature at $t = 480 \text{ s}$ using Euler's method for several step sizes.

3. Put the Differential Equation into Euler Form

Define

$$f(t, \theta) = -2.2067 \times 10^{-12} [\theta^4 - 81 \times 10^8]$$

so that Euler's method becomes

$$\theta(i+1) = \theta(i) + f(t(i), \theta(i)) h$$

and time advances according to

$$t(i+1) = t(i) + h.$$

4. Supplied Worked Reference for $h = 240 \text{ s}$

For $i = 0$:

$$t_0 = 0, \theta_0 = 1200$$

$$\theta_1 = \theta_0 + f(t_0, \theta_0)h$$

$$= 1200 + [-2.2067 \times 10^{-12}(1200^4 - 81 \times 10^8)] \times 240$$

$$= 1200 + (-4.5579)(240) \text{ approximately } 106.09 \text{ K}$$

Therefore, the supplied approximation is

$\theta(240)$ approximately 106.09 K.

For $i = 1$:

$$\begin{aligned}\theta_2 &= \theta_1 + f(t_1, \theta_1)h \\ &= 106.09 + [-2.2067 \times 10^{-12}(106.09^4 - 81 \times 10^8)] \times 240 \\ &= 106.09 + (0.017595)(240) \text{ approximately } 110.32 \text{ K}\end{aligned}$$

and hence

$\theta(480)$ approximately 110.32 K.

Checkpoint: your implementation for $h = 240$ s should reproduce these supplied values to reasonable numerical precision.

5. Supplied Table 2

Step size h (s)	$\theta(480)$ (K)	ET
480	-987.81	1635.4
240	110.32	537.26
120	546.77	100.80
60	614.97	32.607
30	632.77	14.806

Use the step sizes in this table when carrying out the numerical study.

6. Task 1 - Implement Euler's Method in Python

Write a Python program that implements Euler's method for each of the following step sizes:

$h = 480, 240, 120, 60, 30$ seconds

For every step size, begin with $t = 0$ and $\theta = 1200$ K and advance the numerical solution until $t = 480$ s.

Suggested algorithm:

1. Define $f(t, \theta)$.
2. Choose a step size h .
3. Set $t = 0$ and $\theta = 1200$.
4. Store the initial time and temperature.
5. While $t < 480$:
 - calculate θ_{new} using Euler's formula
 - calculate $t_{\text{new}} = t + h$
 - store the new values
 - update t and θ
6. Report θ at $t = 480$.
7. Repeat for the remaining step sizes.

Do not type the supplied table values into your program as calculated results.

Python Syntax Hints

```
def f(t, theta):
    # Return d(theta)/dt
    ...

t = 0.0
theta = 1200.0

while t < 480:
    # Euler update
    ...
    # Advance time
    ...
```

Complete the missing statements yourself from the mathematical formula.

7. Task 2 - Write the Results to a File

Your numerical results must be written to a plain-text data file so that Gnuplot can read them.

A suitable column structure is:

#	time(s)	Euler_temperature(K)	reference_temperature(K)
0	...	...	...
30	...	...	...
60	...	...	...
...	...	...	...
480	...	...	...

Use whitespace-separated columns.

Important note: Table 2 supplies $\theta(480)$ and ET for each step size, but it does not supply a complete exact/reference temperature-versus-time series. If the required second curve is an exact temperature curve over time, use the reference dataset supplied separately by the instructor. Do not invent missing exact values.

File-Writing Hint

```
with open("examNumber_results.dat", "w") as file:
    file.write("# time Euler reference\n")

    # Write one numerical row for each time point.
    # file.write(...)
```

8. Task 3 - Write the Gnuplot Script

Create a Gnuplot script that reads your data file and produces **two curves on the same plot**.

The Gnuplot file must be named:

examNumber_plot.gp

Replace examNumber with your own examination number.

The chart should contain a descriptive title, Time (s) on the x-axis, Temperature (K) on the y-axis, a legend, and two clearly identified curves.

Gnuplot Syntax Hints

```
set title "...
set xlabel "Time (s)"
set ylabel "Temperature (K)"
set grid

plot "..." using 1:2 with linespoints title "...", \
    "..." using 1:3 with lines title "..."
```

using 1:2 means column 1 supplies x and column 2 supplies y.

9. Suggested Project Structure

```
examNumber_task2/
|
|-- examNumber_euler.py
|-- examNumber_results.dat
|-- examNumber_plot.gp
`-- examNumber_plot.png      # if the plot is saved
```

10. Checkpoints

Before completing the project, check the following:

- For $h = 240$ s, the program gives approximately $\theta(240) = 106.09$ K.
- For $h = 240$ s, the program gives approximately $\theta(480) = 110.32$ K.
- Every simulation finishes at exactly $t = 480$ s.
- All requested step sizes have been tested.
- The data file contains consistent numerical columns.

- The Gnuplot script reads the correct data file.
- Both required curves appear on the same graph.

11. Interpretation

The supplied table shows that the Euler approximation changes considerably as the step size changes. A very large step can produce a poor or even physically unrealistic approximation. Your interpretation should discuss why reducing h generally improves the numerical representation of the changing slope.

12. Required Deliverables

- Python source file implementing Euler's method.
- Numerical data/results file used for plotting.
- Gnuplot script named examNumber_plot.gp.
- Final plot containing two curves on the same axes.
- A short interpretation of the effect of step size on the approximation.

13. Assessment Checklist

Component	Expected evidence
Euler implementation	Correct function, initial condition, update formula and iteration
Step-size study	$h = 480, 240, 120, 60$ and 30 s
File I/O	Numerical results written in a Gnuplot-readable format
Gnuplot script	Correct filename and two curves on one plot
Presentation	Title, axes, units and legend are clear
Interpretation	Effect of step size is discussed
Reproducibility	Submitted files run from the project folder

Submission principle: the numerical values in the submitted data and plot must be produced by the student's Python implementation.